

Лабораторная работа №14

Программирование в командном процессоре ОС UNIX. Расширенное программирование

Иванова Анастасия Сергеевна

2026-05-12

1. Докладчик
2. Вводная часть
3. Выполнение лабораторной работы
4. Контрольные вопросы
5. Вывод

Раздел 1

1. Докладчик

1. Докладчик

- Иванова Анастасия Сергеевна



1. Докладчик

- Иванова Анастасия Сергеевна
- 1 курс группа НКАбд-07-25



1. Докладчик

- Иванова Анастасия Сергеевна
- 1 курс группа НКАбд-07-25
- Российский университет дружбы народов



1. Докладчик

- Иванова Анастасия Сергеевна
- 1 курс группа НКАбд-07-25
- Российский университет дружбы народов
- 1132250427@rudn.ru



Раздел 2

2. Вводная часть

- Расширенное программирование в `bash` позволяет решать сложные задачи автоматизации

- Расширенное программирование в `bash` позволяет решать сложные задачи автоматизации
- Механизмы синхронизации процессов необходимы при разработке многопоточных скриптов

- Расширенное программирование в `bash` позволяет решать сложные задачи автоматизации
- Механизмы синхронизации процессов необходимы при разработке многопоточных скриптов
- Создание аналогов системных утилит (`man`) углубляет понимание работы ОС

- Расширенное программирование в `bash` позволяет решать сложные задачи автоматизации
- Механизмы синхронизации процессов необходимы при разработке многопоточных скриптов
- Создание аналогов системных утилит (`man`) углубляет понимание работы ОС
- Генерация случайных данных полезна для тестирования и моделирования

Объект и предмет исследования

Объект исследования: - Командный процессор bash - Механизмы синхронизации процессов (файловые блокировки) - Встроенные переменные и функции оболочки

Предмет исследования: - Реализация семафоров на уровне файловой системы - Создание аналога команды map - Генерация случайных последовательностей с использованием \$RANDOM
- Взаимодействие между несколькими процессами

- Изучить основы программирования в оболочке ОС UNIX

- Изучить основы программирования в оболочке ОС UNIX
- Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов

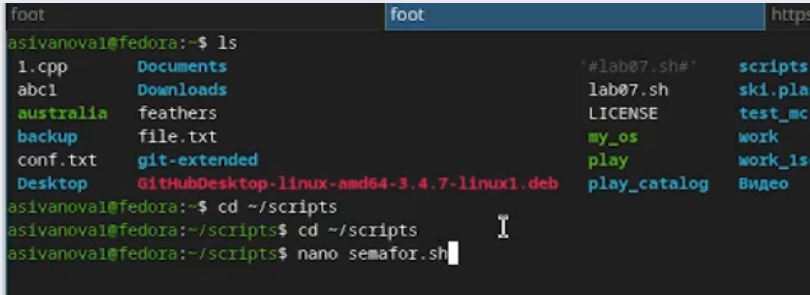
Раздел 3

3. Выполнение лабораторной работы

Задание 1. Реализация упрощённого механизма семафоров

Создание файла semafor.sh

nano semafor.sh



The screenshot shows a terminal window with a dark background. At the top, there are three tabs labeled 'foot', 'foot', and 'https'. The terminal content shows a user named 'asivanova' at a 'fedora' prompt. They run 'ls' and see a list of files and directories. Then they run 'cd ~/scripts' and 'cd ~/scripts' again. Finally, they run 'nano semafor.sh' and the cursor is at the end of the command line.

```
foot foot https
asivanova@fedora:~$ ls
1.cpp      Documents  '#lab07.sh#'  scripts
abc1       Downloads  lab07.sh      ski.pla
australia  feathers   LICENSE       test_mc
backup     file.txt   my_os         work
conf.txt   git-extended  play         work_ls
Desktop    GitHubDesktop-linux-amd64-3.4.7-linux1.deb  play_catalog  Видео
asivanova@fedora:~$ cd ~/scripts
asivanova@fedora:~/scripts$ cd ~/scripts
asivanova@fedora:~/scripts$ nano semafor.sh
```

Рисунок 1: Создание файла semafor.sh

Листинг semafor.sh

```
#!/bin/bash
```

```
LOCK_FILE="/tmp/semaphore.lock"
```

```
t1=5
```

```
t2=3
```

```
while true; do
```

```
    if ln "$LOCK_FILE" "$LOCK_FILE.lock" 2>/dev/null; then
```

```
        echo "$$: ресурс захвачен, использую в течение $t2 секунд"
```

```
        sleep $t2
```

```
        rm "$LOCK_FILE.lock"
```

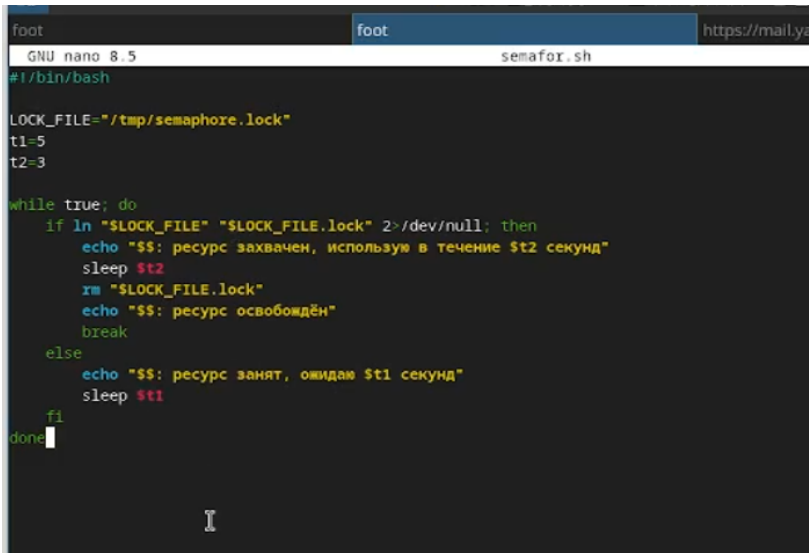
```
        echo "$$: ресурс освобождён"
```

```
        break
```

```
    else
```

```
        echo "$$: ресурс занят, ожидаю $t1 секунд"
```

```
        sleep $t1
```



The image shows a terminal window with a dark background. At the top, there are three tabs: 'foot', 'foot', and 'https://mail.ya...'. The active tab is 'foot'. Below the tabs, the text 'GNU nano 8.5' is on the left and 'semaphor.sh' is on the right. The main area of the terminal displays the following script content:

```
#!/bin/bash

LOCK_FILE="/tmp/semaphore.lock"
t1=5
t2=3

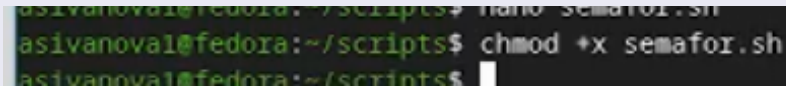
while true; do
    if ln "$LOCK_FILE" "$LOCK_FILE.lock" 2>/dev/null; then
        echo "$$: ресурс захвачен, использую в течение $t2 секунд"
        sleep $t2
        rm "$LOCK_FILE.lock"
        echo "$$: ресурс освобождён"
        break
    else
        echo "$$: ресурс занят, ожидаю $t1 секунд"
        sleep $t1
    fi
done
```

A cursor is visible at the end of the 'done' line.

Рисунок 2: Листинг semaphore.sh

Делаем скрипт исполняемым

```
chmod +x semafor.sh
```



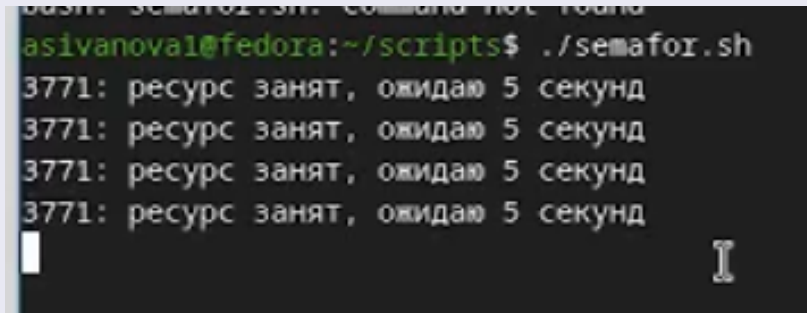
A terminal window screenshot showing the command `chmod +x semafor.sh` being executed. The prompt is `asivanova1@fedora:~/scripts$`. The command is entered on the second line, and the prompt returns on the third line.

Рисунок 3: Изменение прав доступа

Тестирование работы семафора

Терминал 1:

```
./semafor.sh
```

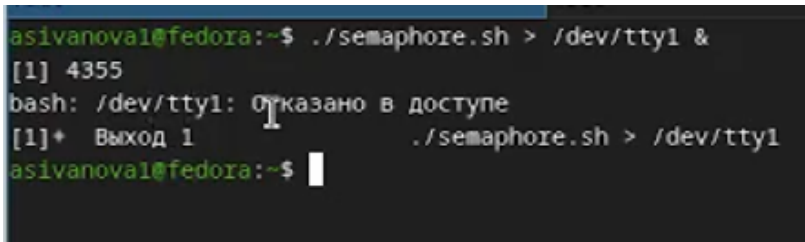


```
asivanova1@fedora:~/scripts$ ./semafor.sh
3771: ресурс занят, ожидаю 5 секунд
3771: ресурс занят, ожидаю 5 секунд
3771: ресурс занят, ожидаю 5 секунд
3771: ресурс занят, ожидаю 5 секунд
```

Рисунок 4: Тестирование семафора терминал 1

Терминал 2 (фоновый запуск с перенаправлением вывода):

```
./semafor.sh > /dev/tty1 &
```



```
asivanova1@fedora:~$ ./semaphore.sh > /dev/tty1 &
[1] 4355
bash: /dev/tty1: Отказано в доступе
[1]+  Выход 1                  ./semaphore.sh > /dev/tty1
asivanova1@fedora:~$
```

Рисунок 5: Тестирование семафора терминал 2

Задание 2. Реализация команды man

Создание файла `my_man.sh`

```
nano my_man.sh
```

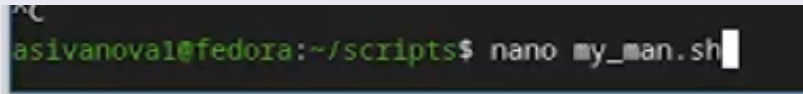


Рисунок 6: Создание файла `my_man.sh`

Листинг my_man.sh

```
#!/bin/bash

if [ $# -ne 1 ]; then
    echo "Использование: $0 <команда>"
    exit 1
fi

CMD="$1"
MAN_FILE="/usr/share/man/man1/${CMD}.1.gz"

if [ -f "$MAN_FILE" ]; then
    zcat "$MAN_FILE" | less
else
    echo "Справка для команды '$CMD' не найдена"
    exit 1
fi
```

```
GNU nano 8.5 my_man.sh
#!/bin/bash

if [ $# -ne 1 ]; then
    echo "Использование: $0 <команда>"
    exit 1
fi

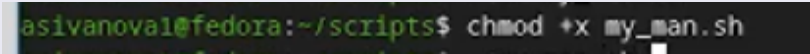
CMD="$1"
MAN_FILE="/usr/share/man/man1/${CMD}.1.gz"

if [ -f "$MAN_FILE" ]; then
    zcat "$MAN_FILE" | less
else
    echo "Справка для команды '$CMD' не найдена"
    exit 1
fi
```

Рисунок 7: Листинг my_man.sh

Делаем скрипт исполняемым

```
chmod +x my_man.sh
```



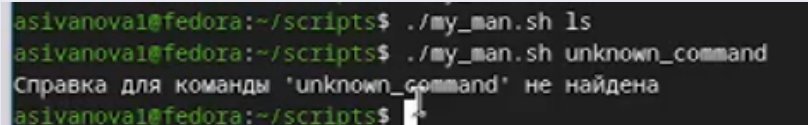
```
asivanova1@fedora:~/scripts$ chmod +x my_man.sh
```

Рисунок 8: Изменение прав доступа

Тестирование работы my_man.sh

```
./my_man.sh ls
```

```
./my_man.sh unknown_command
```



```
asivanova1@fedora:~/scripts$ ./my_man.sh ls
asivanova1@fedora:~/scripts$ ./my_man.sh unknown_command
Справка для команды 'unknown_command' не найдена
asivanova1@fedora:~/scripts$
```

Рисунок 9: Тестирование my_man.sh

Задание 3. Генерация случайной последовательности букв

Создание файла random_letters.sh

```
nano random_letters.sh
```

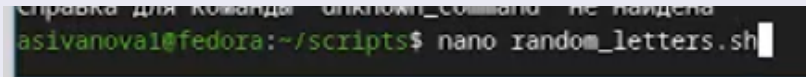


Рисунок 10: Создание файла random_letters.sh

Листинг random_letters.sh

```
#!/bin/bash
```

```
LENGTH=${1:-20}
```

```
for ((i=0; i<LENGTH; i++)); do
```

```
    RAND=$((RANDOM % 26))
```

```
    LETTER=$(printf "\\$(printf '%03o' $((97 + RAND)))
```

```
    echo -n "$LETTER"
```

```
done
```

```
echo
```

```
GNU nano 8.5 random_letters.sh
#!/bin/bash

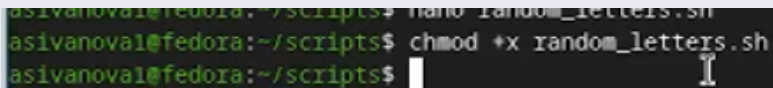
LENGTH=${1:-20}

for ((i=0; i<LENGTH; i++)); do
    RAND=$((RANDOM % 26))
    LETTER=$(printf \\$(printf '%03o' $((97 + RAND))))
    echo -n "$LETTER"
done
echo
```

Рисунок 11: Листинг random_letters.sh

Делаем скрипт исполняемым

```
chmod +x random_letters.sh
```

A terminal window screenshot showing a user named 'asivanova1' at a 'fedora' machine in the directory '~/scripts'. The user has just executed the command 'chmod +x random_letters.sh'. The prompt is now 'asivanova1@fedora:~/scripts\$' with a cursor at the end.

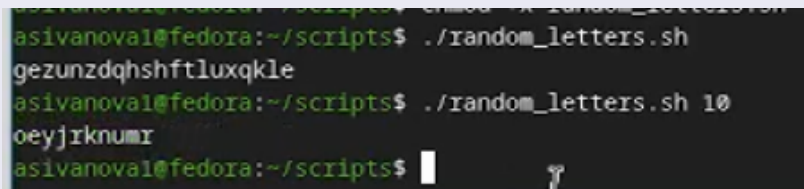
```
asivanova1@fedora:~/scripts$ nano random_letters.sh
asivanova1@fedora:~/scripts$ chmod +x random_letters.sh
asivanova1@fedora:~/scripts$
```

Рисунок 12: Изменение прав доступа

Тестирование работы random_letters.sh

```
./random_letters.sh
```

```
./random_letters.sh 10
```

A screenshot of a terminal window with a dark background and green text. The prompt is 'asivanova1@fedora:~/scripts\$'. The first command is './random_letters.sh' which outputs 'gezunzdqhshftluxqkle'. The second command is './random_letters.sh 10' which outputs 'oeyjrknumr'. The cursor is at the end of the second command line.

```
asivanova1@fedora:~/scripts$ ./random_letters.sh
gezunzdqhshftluxqkle
asivanova1@fedora:~/scripts$ ./random_letters.sh 10
oeyjrknumr
asivanova1@fedora:~/scripts$
```

Рисунок 13: Тестирование random_letters.sh

Раздел 4

4. Контрольные вопросы

4. Контрольные вопросы

1. Найдите синтаксическую ошибку в следующей строке:

```
while [$1 != "exit"]
```

Ответ: пропущены пробелы после [и перед]. Правильно: `while [ "$1" != "exit" ]`

2. Как объединить (конкатенация) несколько строк в одну? Ответ: можно использовать оператор +=:

```
str="Hello"  
str+=" World"
```

3. Найдите информацию об утилите seq. Какими иными способами можно реализовать её функционал? Ответ: seq генерирует последовательность чисел. Альтернативы:

```
for i in {1..10}; do echo $i; done  
for ((i=1; i<=10; i++)); do echo $i; done
```

4. Какой результат даст вычисление выражения $\$(10/3)$? Ответ: 3 (целочисленное деление).

5. Укажите кратко основные отличия командной оболочки zsh от bash. Ответ: zsh имеет автодополнение с подсветкой, поддержку ассоциативных массивов, разделяемые истории между сеансами.

6. Проверьте, верен ли синтаксис данной конструкции:

```
for ((a=1; a <= LIMIT; a++))
```

Ответ: синтаксис верен.

7. Сравните язык `bash` с какими-либо языками программирования. Какие преимущества у `bash` по сравнению с ними? Какие недостатки? Ответ:

Преимущества: простота автоматизации команд ОС, встроенные инструменты для работы с файлами и процессами.

Недостатки: медленная работа, ограниченная поддержка чисел с плавающей точкой.

Раздел 5

5. Вывод

5. Вывод

В ходе лабораторной работы были разработаны три скрипта:

- `semafor.sh` – реализация упрощённого механизма семафоров с использованием файловой блокировки.

Приобретены навыки межпроцессного взаимодействия через файловые блокировки, работы со сжатыми map-страницами (`zcat`), генерации случайных последовательностей, а также изучены особенности синтаксиса `bash`.

5. Вывод

В ходе лабораторной работы были разработаны три скрипта:

- `semafor.sh` – реализация упрощённого механизма семафоров с использованием файловой блокировки.
- `my_man.sh` – аналог команды `man` для просмотра справки из `/usr/share/man/man1`.

Приобретены навыки межпроцессного взаимодействия через файловые блокировки, работы со сжатыми `man`-страницами (`zcat`), генерации случайных последовательностей, а также изучены особенности синтаксиса `bash`.

5. Вывод

В ходе лабораторной работы были разработаны три скрипта:

- `semafor.sh` – реализация упрощённого механизма семафоров с использованием файловой блокировки.
- `my_man.sh` – аналог команды `man` для просмотра справки из `/usr/share/man/man1`.
- `random_letters.sh` – генерация случайной последовательности букв латинского алфавита с использованием `$RANDOM`.

Приобретены навыки межпроцессного взаимодействия через файловые блокировки, работы со сжатыми `man`-страницами (`zcat`), генерации случайных последовательностей, а также изучены особенности синтаксиса `bash`.

